HNC-Benchmark TODO 开发方案

供 VSCode + Claude Code 使用

版本：v3.0 | 2026-04-04（角色调整：开发者只负责搭建框架和编写脚本，数据由他人运行）

读取顺序：先读 hnc-benchmark-context-v5.md，再读本文件

重要背景说明

开发者角色：负责搭建整个项目框架、编写所有脚本和程序。 **数据运行角色**：由其他人负责下载 Hancock 数据、运行脚本、生成数据集。

因此所有脚本必须满足以下要求：

- 数据路径通过**命令行参数或配置文件**传入，不硬编码
- 提供 `--dry-run` 模式或内置 **mock 数据**，使开发者无数据也能验证逻辑
- 有清晰的 **README / docstring** 说明输入输出格式和运行方法
- 脚本之间接口清晰，可独立运行，也可被 `case_builder.py` 串联调用

当前开发优先级

```
立即可开始（完全不需要数据）：
  P0-2 ~ P0-4 的所有脚本逻辑
  P0-3 NCCN 路径规则引擎 + 单元测试
  P0-6 Case 目录结构定义 + JSON Schema
  PHASE 1 评测引擎全部
  PHASE 2 网站全部

需要等数据的（由他人运行）：
  P0-2 实际跑出字段完整性统计结果
  P0-5 验证 WSI patch 提取效果
  P0-6 生成真实 Case JSON 和专家标注工作表
  P0-7 数据集质量验证
```

PHASE 0：数据处理脚本（开发框架，由他人运行）

P0-1 环境说明文档

```
[ ] 编写 README.md (项目根目录)
    内容:
    - 项目概述 (一段话)
    - 环境依赖安装:
        Python >= 3.10
        pip install openslide-python Pillow numpy pandas anthropic tqdm openpyxl
        Linux: apt-get install openslide-tools
    - Hancock 数据集获取方式:
        URL: https://www.cancerimagingarchive.net/collection/hancock/
        需要 IBM Aspera Connect 插件 (4.5TB总量)
        按需下载策略 (StructuredData → TextData → WSI → GeoJSON)
    - 各脚本运行顺序说明
    - 数据目录结构约定

[ ] 编写 config.example.yaml (配置文件模板)
    内容:
    hancock_root: "/path/to/HANCOCK"          # Hancock 数据根目录
    output_root: "./dataset"                   # 生成的 Case 数据集输出目录
    expert_review_root: "./expert_review"      # 专家审核文件输出目录
    anthropic_api_key: "sk-..."                # 用于 LLM Judge / evidence 提取
    mock_mode: false                           # true = 使用内置 mock 数据跑通流程
```

P0-2 结构化数据解析与字段映射

```
[ ] parse_hancock.py
    功能: 解析 clinical_data.json + pathological_data.json, 输出字段完整性报告

    运行方式:
      python data_processing/parse_hancock.py \
        --hancock-root /path/to/HANCOCK \
        --output completeness_report.csv \
        --dry-run  # 使用内置5条mock数据验证脚本逻辑

    重点检查字段:
      cT_stage, cN_stage (直接作为 Step 1 输入, 必须完整)
      pT_stage, pN_stage, resection_status, perinodal_invasion,
      lymphovascular_invasion_L, vascular_invasion_V,
      perineural_invasion_Pn, grading_hpv, infiltration_depth_in_mm,
      closest_resection_margin_in_cm, histologic_type,
      adjuvant_radiotherapy, adjuvant_systemic_therapy,
      adjuvant_chemoradiotherapy, adjuvant_systemic_therapy_mode

    输出: completeness_report.csv (字段名, 总数, 缺失数, 缺失率, 示例值)

    mock 数据: 脚本内置5条假患者数据, --dry-run 时不读取真实文件

[ ] classify_larynx_icd.py
    功能: 根据 ICD codes 区分声门喉癌 vs 声门上喉癌

    运行方式:
```

```
      python data_processing/classify_larynx_icd.py \
        --input clinical_data.json \
        --output icd_larynx_review.csv \
        --dry-run

    ICD规则:
      C32.0 → glottic (声门喉癌)
      C32.1 → supraglottic (声门上喉癌)
      C32.2 → subglottic (声门下, 罕见)
      C32.9 → NOS (需专家人工确认, 输出至审核列表)

    输出:
      - cancer_subtype 字段写入各患者 meta.json
      - icd_larynx_review.csv (C32.9 NOS 案例列表, 供专家确认)

[ ] detect_data_issues.py
    功能: 检测数据矛盾案例

    运行方式:
      python data_processing/detect_data_issues.py \
        --input pathological_data.json clinical_data.json \
        --output data_contradiction_list.csv \
        --dry-run

    检测逻辑:
      - adjuvant_systemic_therapy=no 但 adjuvant_systemic_therapy_mode 有值
      - adjuvant 字段全为 no 但 adjuvant_treatment_intent=curative

    输出: data_contradiction_list.csv (patient_id, 矛盾字段, 矛盾描述)

[ ] detect_cTcN_leak.py
    功能: 扫描手术报告和病史文本, 检测是否含有 cT/cN 明确描述

    运行方式:
      python data_processing/detect_cTcN_leak.py \
        --text-dir /path/to/HANCOCK/TextData \
        --output leak_list.csv \
        --dry-run  # 使用内置含关键词的假文本验证

    关键词: cT1, cT2, cT3, cT4, cN0, cN1, cN2, cN3, 临床分期, clinical staging

    输出:
      - leak_list.csv (patient_id, 文件路径, 泄露片段)
      - 同时更新 meta.json: contains_cTcN_leak: true/false

    说明: cT/cN 的值本身直接来自 pathological_data.json, 检测到泄露只是标记
          该案例为"信息更新能力测试案例", 泄露内容由人工从对应文本中删去
```

P0-3 NCCN 路径节点自动标注

```
[ ] nccn_router.py
    功能：根据分期和癌种输出 NCCN 路径节点（纯规则，完全不需要数据）

    接口：
      from data_processing.nccn_router import NCCNRouter
      router = NCCNRouter()
      node = router.route(
          tumor_site="Hypopharynx",
          cT="cT3", cN="cN2b", M="M0",
          p16_status=None  # 口咽癌才需要
      )
      # 返回："HYPO-3"

    规则表（严格按照 context-v5.md 第6节，不得自行修改）：
      口腔癌:     T1-2N0→OR-2; T3N0/T1-3N1-3/T4aN0-3→OR-3;
                  T4b/无法切除→ADV-1; M1→ADV-2
      口咽癌p16-: T1-2N0-1→ORPH-2; T3-4aN0-1→ORPH-3;
                  T1-4aN2-3→ORPH-4; T4b→ADV-1; M1→ADV-2
      口咽癌p16+: M1→ADV-2; T1-2N0→ORPHPV-1; T0-2N1(≤3cm)→ORPHPV-2;
                  T0-2N1(>3cm或≥2个≤6cm)/T1-2N2/T3N0-2→ORPHPV-3;
                  T0-3N3/T4N0-3→ORPHPV-4
      下咽癌:     适合喉保留(多T1N0,部分T2N0)→HYPO-2; T1-3N0-3→HYPO-3;
                  T4aN0-3→HYPO-5; T4b/无法切除/不适合手术→ADV-1; M1→ADV-2
      声门喉癌:   原位癌/适合保留(T1-2N0,选定T3N0)→GLOT-2;
                  T3全喉N0-1→GLOT-3; T3全喉N2-3→GLOT-4;
                  T4a→GLOT-6; T4b→ADV-1; M1→ADV-2
      声门上喉癌: 适合保留(多T1-2N0,部分T3)→SUPRA-2; T3N0全喉→SUPRA-3;
                  T4aN0→SUPRA-8; N+→SUPRA-4分流(T1-2N+/部分T3N1→SUPRA-5;
                  多数T3N1-3→SUPRA-6; T4aN1-3→SUPRA-8); T4b→ADV-1; M1→ADV-2

    [ ] 配套单元测试 tests/test_nccn_router.py
        覆盖：每个癌种的典型分期、边界案例（T3/T4a）、M1、ADV分支
        运行：pytest tests/test_nccn_router.py -v
        不需要任何数据文件，纯逻辑测试

[ ] adjuvant_router.py
    功能：根据病理特征输出辅助治疗指南建议（纯规则）

    接口：
      from data_processing.adjuvant_router import AdjuvantRouter
      router = AdjuvantRouter()
      result = router.route(
          nccn_node="HYPO-3",
          pT="pT4a", pN="pN2b",
          perinodal_invasion="yes",
          resection_status="R0",
          closest_margin_cm=0.08,
          lvi_L="yes", pni_Pn="no"
      )
      # 返回：{
      #   "guideline_adjuvant_chemoRT": "yes",
      #   "recommendation_category": "1类",
      #   "adjuvant_decision_trigger": ["perinodal_invasion", "margin<0.1cm"]
```

```
      # }

    规则严格按照 context-v5.md 第7节各类型决策流

    [ ] 配套单元测试 tests/test_adjuvant_router.py
        覆盖：高危（ENE/R1）、其他风险、无不良特征三类；各癌种节点
```

P0-4 手术报告清洗

```
[ ] clean_surgery_reports.py
    功能：检测手术报告中医生直接给出辅助治疗建议的段落，输出待人工删除列表

    运行方式：
      python data_processing/clean_surgery_reports.py \
        --text-dir /path/to/HANCOCK/TextData/reports_english \
        --output adjuvant_suggestion_list.csv \
        --dry-run  # 使用内置含"adjuvant"关键词的假报告验证

    检测模式（不自动删除，仅标记）：
      - 英文：adjuvant, recommend radiation, postoperative chemo...
      - 中文（若有译文）：建议术后放疗、推荐辅助化疗...

    输出：adjuvant_suggestion_list.csv（patient_id，文件路径，匹配段落）
    由人工确认后手动删去相关内容
```

P0-5 WSI Patch 提取

```
[ ] extract_wsi_patches.py
    功能：从 SVS 格式 WSI 中提取带比例尺的 patch 图像

    运行方式：
      python data_processing/extract_wsi_patches.py \
        --wsi-dir /path/to/HANCOCK/WSI_PrimaryTumor \
        --geojson-dir /path/to/HANCOCK/WSI_PrimaryTumor_Annotations \
        --ln-wsi-dir /path/to/HANCOCK/WSI_LymphNode \
        --output-dir ./dataset/cases \
        --patient-ids 0001 0002 0003  # 指定患者，不指定则处理全部
        --dry-run  # 跳过实际 openslide 调用，只验证目录结构和参数

    输出（每个患者 step3_images/ 目录下）：
      primary_5x_scalebar.jpg     （全景，target_mpp=2.0μm/px，比例尺100μm）
      primary_20x_scalebar.jpg    （细节，target_mpp=0.5μm/px，比例尺50μm）
      lymphnode_10x_scalebar.jpg  （淋巴结，target_mpp=1.0μm/px，比例尺100μm）

    Tier1（有GeoJSON标注时）：读取多边形坐标 → 计算外接矩形+10%边距 → read_region()
    Tier2（无标注时 fallback）：缩略图检测组织区域 → 从高密度区采样6个tile

    比例尺绘制：
      bar_px = int(100 / target_mpp)  # 100μm 对应像素数
```

```
      右下角绘制白色矩形 + "100μm" 黑色文字

[ ] record_ln_wsi_status.py
    功能：记录每个患者的淋巴结 WSI 可用状态

    运行方式：
      python data_processing/record_ln_wsi_status.py \
        --ln-wsi-dir /path/to/HANCOCK/WSI_LymphNode \
        --pathological-data /path/to/pathological_data.json \
        --dataset-dir ./dataset/cases \
        --dry-run

    规则：
      pN0 → ln_wsi_missing_reason: "pN0" (无需WSI，评分时输出 not_applicable)
      pN+ 且文件不存在 → ln_wsi_missing_reason: "specimen_unavailable"
    写入各患者 meta.json
```

P0-6 完整 Case JSON 生成

```
[ ] case_builder.py
    功能：串联 P0-2 ~ P0-5 的所有输出，为每个患者生成完整 Case 目录结构
    这是主入口脚本，其他脚本可单独调用，也可通过此脚本统一调度

    运行方式：
      python data_processing/case_builder.py \
        --config config.yaml \
        --patient-ids 0001 0002 ...  # 不指定则处理全部
        --dry-run  # 使用 mock 数据完整走通流程，验证目录和 JSON 结构

    生成目录结构：
    dataset/cases/Case_{patient_id}/
    ├── meta.json
    ├── step1_input.json     # 患者基本信息 + 直接给出的 cT/cN
    ├── step1_gt.json        # NCCN路径GT (规则引擎生成)
    ├── step2_input.json     # 手术报告 (已清洗) + 病史 + 淋巴结数量
    ├── step2_gt.json        # 初步pT/R/histotype GT + evidence anchors (AI提取，待
审核)
    ├── step3_images/
    │   ├── primary_5x_scalebar.jpg
    │   ├── primary_20x_scalebar.jpg
    │   └── lymphnode_10x_scalebar.jpg  # 或缺失标记文件
    ├── step3_gt.json        # 微观病理GT (直接来自 pathological_data.json)
    └── step4_gt.json        # 辅助治疗GT (数据集 + 规则引擎 + 专家修正标记)

    step1_input.json 字段：
      year_of_initial_diagnosis, age_at_initial_diagnosis, sex, smoking_status,
      primarily_metastasis, primary_tumor_site, icd_code,
      cT_stage (来自 pathological_data.json) ,
      cN_stage (来自 pathological_data.json) ,
      medical_history (来自 histories_english/，泄露内容已删去)
      【不包含】hpv_association_p16, adjuvant_treatment_intent
```

```
    step1_gt.json 字段:
      current_phase: "STAGING_AND_TREATMENT_DECISION"
      cT_stage_confirmed, cN_stage_confirmed
      nccn_pathway_node (nccn_router 生成)
      recommendation_category (null 表示该节点无固定等级)
      recommended_treatment_options (按节点固定文本)
      surgery_detail (按节点固定文本)
      p16_subtask_required: true/false (口咽癌=true)
      p16_subtask_gt: "需要 p16/HPV 检测结果" (口咽癌专用)

    step4_gt.json 字段:
      adjuvant_treatment_intent (来自数据集)
      adjuvant_radiotherapy_y_n, adjuvant_radiotherapy_type,
      adjuvant_systemic_therapy_y_n, adjuvant_systemic_therapy_mode,
      adjuvant_chemoradiotherapy_y_n
      guideline_adjuvant_RT, guideline_adjuvant_chemoRT (adjuvant_router 生成)
      patient_declined (指南与实际不符时标记, 待专家确认)

[ ] generate_expert_review.py
    功能: 生成专家标注工作表 (xlsx) , 汇总所有需要人工审核的项目

    运行方式:
      python data_processing/generate_expert_review.py \
        --dataset-dir ./dataset/cases \
        --output-dir ./expert_review \
        --dry-run

    生成文件:
      evidence_anchor_review.csv    # pT/R/histotype 对应原文短语 (AI提取, 需确认)
      data_contradiction_list.csv   # 矛盾数据 (类型A/B, 需修正)
      icd_larynx_review.csv         # C32.9 NOS 声门归类 (需人工确认)
      nccn_node_sample_review.csv   # 20% 抽样验证路径节点
      adjuvant_deviation_list.csv   # 指南与实际治疗不符案例
      leak_list.csv                 # cTcN 泄露片段 (需人工删除)
```

P0-7 数据集质量验证

```
[ ] validate_dataset.py
    功能: 统计数据集完整性, 生成质量报告, 划分评测集

    运行方式:
      python data_processing/validate_dataset.py \
        --dataset-dir ./dataset/cases \
        --output dataset_stats_report.md \
        --dry-run

    报告内容:
      - 各癌种患者数量
      - cT/cN 字段完整率 (核心输入, 目标接近100%)
      - 每步 GT 字段的完整率
```

```
      - WSI patch 提取成功率
      - 各字段 null 值数量
      - 待专家审核项总数

    注：450例核心评测集的具体数量在实际数据分布统计完成后确定
        目前按 Simple/Medium/Hard 分层抽样，比例待定

    难度分级规则：
      Simple：pT1-2，pN0，无不良特征
      Medium：pT3，pN1，1个不良特征
      Hard：pT4，pN2-3，多个不良特征 or 升期

    输出：
      - dataset_stats_report.md
      - difficulty 字段写入各患者 meta.json
      - benchmark_cases.json（按难度分层的案例ID列表，数量待定）
```

PHASE 1：评测引擎（可立即开始，不需要数据）

P1-1 评分器实现

```
[ ] structured_scorer.py
    功能：实现所有结构化字段的自动评分
    规则严格按照 context-v5.md 第8节

    接口：
      from evaluation.structured_scorer import StructuredScorer
      scorer = StructuredScorer()
      score = scorer.score_field(
          field_name="nccn_pathway_node",
          predicted="HYPO-3",
          ground_truth="HYPO-5"
      )
      # 返回：{"score": 0.5, "reason": "same-site adjacent node"}

    步骤权重：Step1=25%，Step2=20%，Step3=30%，Step4=25%
    每步内部：60%×结构化字段 + 40%×LLM-Judge

    [ ] 单元测试 tests/test_structured_scorer.py
        覆盖各字段类型：分期相邻分、节点相邻分、数值范围、二元字段

[ ] llm_judge.py
    功能：封装 LLM-as-Judge 评分调用

    接口：
      from evaluation.llm_judge import LLMJudge
      judge = LLMJudge(api_key="...", model="claude-sonnet-4-6")
      result = judge.evaluate(
          field_name="nccn_pathway_reasoning",
          predicted_text="...",
```

```
            source_context="...",    # 原始输入（用于核验无幻觉）
            criteria=[
                "内容基于提供的原始信息，无幻觉",
                "逻辑可追溯，结论与依据有因果关系",
                "领域术语使用准确"
            ]
        )
        # 返回: {"criteria_results": [True, True, False], "score": 0.67}

    每字段评估2次取多数，支持 --mock 跳过真实 API 调用

[ ] safety_checker.py
    功能：检测高危错误，输出 CSS 分和 flag 列表

    接口：
        from evaluation.safety_checker import SafetyChecker
        checker = SafetyChecker()
        result = checker.check(predicted_output, ground_truth)
        # 返回: {
        #   "css_score": 0.6,
        #   "flags": ["PERINODAL_MISS", "HALLUCINATION_FLAG"],
        #   "step_penalties": {"step3": 0.3}
        # }

    检测项（对应 context-v5.md 第8节）：
        PERINODAL_MISS    → Step3×0.3; CSS-0.30
        CHEMORT_MISSED    → Step4×0.2; CSS-0.30
        TEMPORAL_VIOLATION → 违规字段清零
        HALLUCINATION_FLAG → evidence字段清零; CSS-0.10
        LN_WSI_HALLUC     → 该字段×0.2; CSS-0.10
        CASCADE_ERROR     → 额外-0.15

[ ] case_evaluator.py
    功能：整合三个评分器，对单个 Case 完整评分

    接口：
        from evaluation.case_evaluator import CaseEvaluator
        evaluator = CaseEvaluator(config)
        report = evaluator.evaluate(
            case_dir="./dataset/cases/Case_0001",
            model_output_dir="./results/gpt4o/Case_0001",
            mode="AR"  # "AR"（级联）或 "IND"（每步注入GT）
        )
```

P1-2 被测模型接口

```
[ ] model_runner.py
    功能：统一多模型调用接口，处理四步流程的输入构造和输出解析

    支持的模型后端：
        - OpenAI (GPT-4o, GPT-4V)
```

```
      - Anthropic (claude-sonnet-4-6)
      - Google (Gemini Pro Vision)
      - 本地模型 (通过 ollama 或 vllm, 兼容 OpenAI API 格式)

    运行方式:
      python evaluation/model_runner.py \
        --model gpt-4o \
        --case-dir ./dataset/cases/Case_0001 \
        --output ./results/gpt-4o/Case_0001 \
        --step 1  # 运行单步, 不指定则运行全部四步
        --mock    # 不调用真实 API, 返回预设的假输出, 用于调试评分器

[ ] wsi_image_encoder.py
    功能: 将 WSI patches 编码为各 API 要求的 base64 格式, 处理大小限制

    注: Claude API 单图上限 5MB, 超出时裁剪或压缩
        Prompt 内最多 20 张图

[ ] prompt_builder.py
    功能: 为每步骤构建标准化 Prompt, 包含所有约束说明

    每步 Prompt 包含:
      - 当前阶段说明 (current_phase 约束)
      - 该步骤的输入数据 (JSON格式)
      - 输出 JSON Schema (严格定义字段名和类型)
      - 关键约束提示 (如: 零幻觉约束、p16 说明、不提供帕博利珠单抗等)
```

P1-3 完整评测流程

```
[ ] run_benchmark.py
    功能: 批量评测主入口

    运行方式:
      python evaluation/run_benchmark.py \
        --model gpt-4o \
        --dataset-dir ./dataset/cases \
        --case-list ./dataset/benchmark_cases.json \
        --output-dir ./results \
        --mode AR \
        --resume   # 断点续跑, 跳过已有结果的 case

    输出: results/{model_name}/{case_id}/
      step{n}_output.json   # 模型原始输出
      step{n}_score.json    # 该步评分结果
      case_report.json      # 完整评分报告

[ ] results_aggregator.py
    功能: 汇总所有 Case 分数, 生成排行榜条目

    运行方式:
      python evaluation/results_aggregator.py \
```

```
        --results-dir ./results/gpt-4o \
        --output leaderboard_entry.json

    输出字段:
      model_name, total_score, step1_score, step2_score,
      step3_score, step4_score, css_score, ar_score, ind_score,
      eval_date, case_count
```

PHASE 2：基础设施（可立即开始，不需要数据）

P2-1 服务器架构

```
架构方案（无需立即部署，先把代码写好）:
  数据存储: 阿里云/腾讯云 OSS/COS
    hnc-benchmark/
    ├── dataset/cases/{case_id}/    # 评测数据集（不含原始WSI）
    ├── results/{model_name}/       # 评测结果
    └── public/leaderboard.json     # 公开排行榜数据

  评测服务器: 按需租用 GPU（按次付费），不需要长期运行
  网站服务器: 1核2GB 轻量服务器即可

[ ] 编写 scripts/upload_dataset.py
    功能: 将本地生成的 dataset/ 上传到 OSS/COS
    支持增量上传（只上传新增或修改的 Case）

[ ] 编写 scripts/upload_results.py
    功能: 将评测结果上传并触发排行榜更新
```

P2-2 公开网站

```
技术栈:
  前端: Next.js + Tailwind CSS（支持静态导出，可本地部署）
  后端: FastAPI（Python）
  数据库: SQLite（排行榜和结果存储）
  容器: Docker + docker-compose（一键部署）

[ ] 网站页面
    /              # 主页: 项目介绍、方法论、论文链接
    /leaderboard   # 排行榜: 模型名|总分|Step1~4|CSS|AR/IND|日期
    /dataset       # 数据集介绍: 统计信息、字段说明、访问申请
    /docs          # 评测方法: 四步流程、评分规则、JSON Schema
    /submit        # 提交评测请求

[ ] 排行榜功能
    - 按总分排序，支持按 Step 分和 CSS 分排序
    - 按模型类型筛选（专业医疗模型 vs 通用模型）
    - 点击模型名展开各步骤详情
```

```
    - 显示 AR Score 和 IND Score 对比

[ ] docker-compose.yml
    一键启动完整网站（含数据库），不依赖任何云服务
    README_deploy.md 说明本地部署步骤
```

PHASE 3：持续迭代（后续）

```
[ ] 扩充评测案例（从剩余763例中按需增加）
[ ] 支持更多模型后端
[ ] 专家反馈收集机制（网站上的反馈入口）
[ ] 定期更新排行榜和数据集版本管理
[ ] 论文写作（方法论 + 实验结果）
```

项目目录结构

```
hnc-benchmark/
├── CLAUDE.md                          # Claude Code 自动读取的项目说明
├── README.md                          # 人类阅读的项目说明和运行指南
├── config.example.yaml                # 配置文件模板（复制后填写路径）
├── hnc-benchmark-context-v5.md        # 项目上下文（Claude读取用）
├── hnc-benchmark-todo-v3.md           # 本TODO文件
│
├── data_processing/                   # PHASE 0: 数据处理脚本
│   ├── parse_hancock.py               # P0-2: 解析结构化数据
│   ├── classify_larynx_icd.py         # P0-2: 声门/声门上区分
│   ├── detect_data_issues.py          # P0-2: 矛盾检测
│   ├── detect_cTcN_leak.py            # P0-2: cTcN泄露检测
│   ├── nccn_router.py                 # P0-3: NCCN路径规则引擎
│   ├── adjuvant_router.py             # P0-3: 辅助治疗决策引擎
│   ├── clean_surgery_reports.py       # P0-4: 报告清洗
│   ├── extract_wsi_patches.py         # P0-5: WSI提取
│   ├── record_ln_wsi_status.py        # P0-5: 淋巴结WSI状态记录
│   ├── case_builder.py                # P0-6: Case生成器（主入口）
│   ├── generate_expert_review.py      # P0-6: 专家审核工作表生成
│   └── validate_dataset.py            # P0-7: 质量验证
│
├── tests/                             # 单元测试（不需要数据即可运行）
│   ├── test_nccn_router.py
│   ├── test_adjuvant_router.py
│   └── test_structured_scorer.py
│
├── evaluation/                        # PHASE 1: 评测引擎
│   ├── prompt_builder.py              # 各步骤 Prompt 构建
│   ├── model_runner.py                # 多模型接口
│   ├── wsi_image_encoder.py           # WSI图像编码
│   ├── structured_scorer.py           # 结构化字段评分
```

```
│   ├── llm_judge.py                    # LLM裁判评分
│   ├── safety_checker.py               # 临床安全惩罚
│   ├── case_evaluator.py               # 单Case完整评分
│   ├── run_benchmark.py                # 批量评测主入口
│   └── results_aggregator.py           # 结果汇总
│
├── website/                            # PHASE 2: 公开网站
│   ├── frontend/                       # Next.js
│   ├── backend/                        # FastAPI
│   ├── docker-compose.yml
│   └── README_deploy.md
│
├── scripts/                            # 辅助脚本
│   ├── upload_dataset.py               # 上传数据集到 OSS/COS
│   └── upload_results.py               # 上传评测结果
│
├── dataset/                            # 生成的评测数据集（由他人运行后产生）
│   ├── cases/
│   │   └── Case_0001/
│   │       ├── meta.json
│   │       ├── step1_input.json
│   │       ├── step1_gt.json
│   │       ├── step2_input.json
│   │       ├── step2_gt.json
│   │       ├── step3_images/
│   │       ├── step3_gt.json
│   │       └── step4_gt.json
│   ├── benchmark_cases.json            # 评测集案例列表（数量待定）
│   └── dataset_stats_report.md
│
└── expert_review/                      # 待专家审核的文件（由他人运行后产生）
    ├── evidence_anchor_review.csv
    ├── data_contradiction_list.csv
    ├── icd_larynx_review.csv
    ├── nccn_node_sample_review.csv
    ├── adjuvant_deviation_list.csv
    └── leak_list.csv
```